ExTrus
Extreme Trust

10월 5주차 공지

목차

1. 개요

2. 프로젝트 목적

3. 기술적 배경

4. 현행 문제

5. 목표

6. 기대효과

개요

최근 모바일 애플리케이션 역공학(reverse engineering)과 코드 탈취 사례가 증가함에 따라, 앱 보호 기술로서 코드 난독화(obfuscation) 의 중요성이 급격히 높아지고 있다.

기존의 ProGuard 및 R8은 컴파일 시점에 난독화를 수행하지만(소스코드 난독화), 이미 배포된 APK나 타 빌드 환경에서는 적용이 제한적이다.

이러한 한계를 보완하기 위해 Obfuscapk는 빌드된 APK를 대상으로 난독화를 수행할 수 있는 오픈소스 도구로 주목받고 있다.

그러나 Obfuscapk는 아직 연구용 수준의 구현으로, 최신 안드로이드 기술 스택(Compose, Hilt, Coroutines 등)에 대한 대응이 부족하며, 일부 플러그인은 안정성 문제를 가진다.

이에 따라 현행 Obfuscapk의 안정성을 고도화하는 프로젝트를 진행한다.

목적

본 프로젝트의 목적은 다음과 같다:

- Obfuscapk의 내부 구조 및 플러그인 체계를 분석
- 최신 안드로이드 기술(Hilt, Compose, DataStore 등)에 대응 가능한 난독화 로직 조정
- 난독화 후 앱의 보안성과 실행 안정성을 동시에 확보하는 프레임워크 제공

기술적 배경

난독화는 코드의 가독성과 분석 용이성을 인위적으로 저하시켜, 역공학이나 디버깅을 어렵게 만드는 기술이다.

대표적인 기법으로는 이름 변경(Name Obfuscation), 문자열 암호화(String Encryption), 제어 흐름 변경(Control Flow Obfuscation) 등이 있다.

기존 난독화 기술의 한계

- ProGuard / R8: 컴파일 시점에만 적용 가능, 빌드 후 APK에는 적용 불가
- DexGuard: 강력하지만 상용 제품으로 라이선스 비용 부담
- Obfuscapk: 오픈소스, 최신 Android SDK 및 Compose 기반 앱에 대한 호환성 미흡

기술적 배경

Obfuscapk 구조 및 동작 원리

•단계별 파이프라인: Decompile → Obfuscate → Rebuild → ReSign

•플러그인 기반 설계: 각 난독화 기법이 독립된 플러그인으로 동작 (Rename, StringEncryption, Arithmetic 등)

•Smali 코드 조작: apktool을 이용해 디컴파일된 smali 코드를 파싱하고 수정 후 재패키징

현행 문제

ExTrus
Extreme Trust

Smali 처리 로직 복잡, obfuscator 별 오류 발생 시 예외 처리 미흡

BundleDecomplier가 없을 때의, 예외처리 부재 등 aab 처리 호환성

Renaming 시 외부 라이브러리, 리플렉션, 매니페스트, 네이티브 심볼 참조 미보장으로 인한

런타임 오류

목표

테스트 앱으로 난독화를 진행하여 검증을 진행

Obfuscapk의 obfuscator를 적용한 후 정상 동작 및 jadx-gui 및 smali를 통해 난독화 정상 여부 확인

테스트 앱 : Now In Android (구글에서 배포하는 오픈소스 샘플앱) (다른 테스트앱이 추가될 수도 있음, 추가 된다면 추후 공지)

선정 이유:

1. 복잡한 구조: 멀티 모듈, DI(Hilt), Compose, Coroutine 등 모듈화 및 여러 오픈소스, 최신 기술이 탑재되어 있음

2. 네트워크·DB·로컬 저장소 존재: 코드/리소스/네트워크/데이터 계층 난독화 테스트 가능

3. Compose 코드 난독화 영향 확인 가능: Obfuscapk이 smali 수준에서 처리하므로 Compose-generated 코드 난독화 결과 분석 가능

4. 실제 앱처럼 작동: 완성도가 높은 오픈소스앱이므로 난독화 후 동작 비교가 쉬움

목표

ExTrus Extreme Trust

주의 할 점

멀티덱스 & Hilt 관련 이슈

• Hilt가 내부적으로 reflection을 사용하기 때문에, Obfuscapk의 Rename 플러그인이 잘못 동작하면

ClassNotFoundException 발생 가능.

Proto DataStore / Serialization

• 자동 생성된 클래스 이름이 바뀌면 데이터 직렬화 실패 가능

Gradle KTS 기반이라 빌드·서명 스크립트 커스터마이징 필요

•Obfuscapk는 apk 완성본 기준으로 동작하므로, Now in Android을 먼저 빌드 → release APK 추출 → Obfuscapk 적용 → 재서명.

• Rebuild – ReSign의 빌드-사이닝 과정을 준수

기대 효과

- 오픈소스 수준의 난독화 도구를 전문 수준으로 고도화
- 난독화에 대한 지식 및 기술 확보
- 최신 Android 기술에 대응 가능한 유연한 난독화 기술 확보
- 학술 연구 및 산업 보안 솔루션 적용 가능

저장소

기본 obfuscapk 코드를 내려 받은 후 프로젝트를 구현합니다.
Obfuscator별로 간단한 설명 주석이 있으니 참고바랍니다.
'자료 공유' 디렉토리에 obfuscapk 분석 결과 (각 주차별 ppt)를 업로드하여 각자의 내용을
공유하여진행해보면 좋겠습니다.

링크: gl.extrus.net에 접속 -> 계정등록 -> 이메일을 학교메일 (sookmyung.ac.kr)로 등록 -> 추후
확인 후 권한 부여 예정

브랜치는 develop 브랜치 사용

result/ 디렉토리 하위에 테스트 앱 apk가 있으니 활용바랍니다.
테스트 진행 시, 본인 역할 외의 obfuscation 옵션도 적용해보면서 확인해보고, 오류나 안되는 점이
발생하면 그런 부분을 서로 공유하면서 진행해주시길 바랍니다.

메인
nowinandroid-debug.apk: Now in Android apk

서브
jni-debug.apk: c/c++ 네이티브 코드를 포함한 간단한 구조의 샘플 apk
app-debug.apk: 간단한 구조의 샘플 apk